

Week 1 Homework Assignment

While Loops: Condition-Based Repetition

Total Points: 100

Name: _____

Class: _____

Important Instructions

- Work on your own, but you may ask family members for hints only if stuck
- Write your code in Python (*using Thonny or any online IDE*)
- For setting up your code files, kindly see [instructions.pdf](#)
- For written answers, use the spaces provided on this sheet
- Show your work for full credit!

1 Part 1: Understanding Loop Types (15 points)

Time estimate: 15 minutes

1.1 Exercise 1.1: When to Use Which Loop (10 points)

Complete this decision table to understand when to use `for` loops vs `while` loops:

Programming Task	for or while?	Why? (Explain your reasoning)
Print "Hello" 10 times		
Keep asking for password until correct		
Calculate sum of numbers 1 to 100		
Play a game until player types "quit"		
Display a multiplication table (1 to 12)		
Roll dice until you get a 6		

1.2 Exercise 1.2: Loop Components Identification (5 points)

For each while loop below, identify the three essential parts. Write your answers in the table:

```
# Loop A
score = 0
while score < 100:
    points = int(input("Enter points earned: "))
    score = score + points
```

```
# Loop B
countdown = 10
while countdown > 0:
    print(countdown)
    countdown = countdown - 1
```

Loop	Initialize (before loop)	Condition (checked)	Update (inside loop)
A			
B			

2 Part 2: Complete the While Loops (15 points)

Time estimate: 20 minutes

2.1 Exercise 2.1: Countdown Timer (7 points)

Complete this countdown timer program. Fill in the blanks to make it work correctly:

```
# Countdown Timer
# This program counts down from a user-specified number to 1

start = int(input("Enter starting number: "))
counter = _____ # Initialize the counter

while _____: # What condition should we check?
    print(f"T-minus {counter}")
    _____ # Update the counter

print("Blast off!")
```

Expected Output (if user enters 5):

```
Enter starting number: 5
T-minus 5
T-minus 4
T-minus 3
T-minus 2
T-minus 1
Blast off!
```

Save this code as week1_ex2_1.py

2.2 Exercise 2.2: Sum Calculator (8 points)

Complete this program that keeps adding numbers until the sum reaches at least 50:

```
# Sum Calculator
# Keep adding numbers until total reaches 50 or more

total = _____ # Initialize total
count = 0 # Track how many numbers entered

while _____: # Continue until total >= 50
    num = int(input("Enter a number: "))
```

```

----- # Add num to total
count = count + 1
print(f"Current total: {total}")

print(f"Final total: {total}")
print(f"You entered {count} numbers")

```

Test your program: What happens if the first number is already 50 or more?

Save this code as `week1_ex2_2.py`

3 Part 3: Trace the Execution (15 points)

Time estimate: 20 minutes

3.1 Exercise 3.1: Memory Diagram Practice (15 points)

Trace through this code and show how the variables change. Draw a memory diagram for each step:

```

num = 15
counter = 0

while num > 5:
    num = num - 3
    counter = counter + 1
    print(f"num = {num}, counter = {counter}")

```

Memory diagram template with three columns: "After Initialization", "After 1st iteration", "After 2nd iteration", "After 3rd iteration", "After 4th iteration". Each column has boxes for 'num' and 'counter' variables with space to write values. Include arrows showing flow between iterations.

Fill in the table:

Step	Value of num	Value of counter	Condition (num > 5)?
Initial	15	0	True
After iteration 1			
After iteration 2			
After iteration 3			
After iteration 4			

Questions:

- How many times does the loop run? ____
- What is the final value of num? ____
- What is the final value of counter? ____
- What will the program print? Write the complete output below:

4 Part 4: Pattern Recognition (15 points)

Time estimate: 20 minutes

4.1 Exercise 4.1: Identify the Pattern (7 points)

Look at this output and write the while loop that produced it:

Output:

```
20
17
14
11
8
5
2
```

```
# Your while loop here:
counter = -----
while -----:
    -----
    -----
```

Save this code as *week1_ex4_1.py*

4.2 Exercise 4.2: Create Your Own Pattern (8 points)

Write a while loop that prints this doubling pattern:

Output:

```
1
2
4
8
16
32
64
128
```

Hint: The loop should stop when the number exceeds 100.

```
# Doubling pattern
value = -----
while -----:
    -----
    -----
```

Challenge question: How many times does your loop run? ____

Save this code as *week1_ex4_2.py*

5 Part 5: Debug Detective (15 points)

Time estimate: 25 minutes

Each code snippet has errors. Find and fix ALL errors!

5.1 Snippet 1: Missing Initialization (5 points)

```
# This code has 1 error - find it!
# Should count from 1 to 5

while count <= 5:
```

```
print(count)
count = count + 1
```

What's wrong? _____

Fixed version:

Write the corrected code here:

Save your corrected code as *week1-ex5-1-fixed.py*

5.2 Snippet 2: Infinite Loop Alert! (5 points)

This code has 1 error - it never stops!
Should print numbers 5, 4, 3, 2, 1

```
countdown = 5
while countdown > 0:
    print(countdown)
print("Done!")
```

Diagram showing infinite loop: Initialize counter = 5 →
Check condition (5 > 0) True → Print 5 → Check
condition (5 > 0) True → Print 5 → ... with circular arrow
indicating loop never ends because counter never changes

What's wrong? _____

Fixed version:

Write the corrected code here:

Save your corrected code as *week1-ex5-2-fixed.py*

5.3 Snippet 3: Wrong Condition (5 points)

This code has 1 error - condition is wrong!
Should keep asking until user types "yes"

```
answer = ""
while answer == "yes":
    answer = input("Do you want to continue? ")
print("Thank you!")
```

What's wrong? _____

Trace it: If this code runs, what happens immediately? _____

Fixed version:

```
# Write the corrected code here:
```

```
-----  
-----  
-----  
-----
```

Save your corrected code as `week1_ex5_3_fixed.py`

6 Part 6: Creative Coding Challenge (20 points)

Time estimate: 30 minutes

Choose ONE of the following projects and complete it:

6.1 Option A: Password Validator (20 points)

Create a program that keeps asking for a password until the user enters the correct one. After 3 wrong attempts, the program should lock them out.

Requirements for Password Validator

Your program should:

- Set a secret password (you choose it)
- Use a while loop to keep asking for password
- Count the number of attempts
- Stop after 3 wrong attempts OR when correct password is entered
- Display appropriate messages for success or lockout

Example Run 1 (Success):

```
Enter password: hello  
Wrong password! Try again.  
Enter password: python  
Wrong password! Try again.  
Enter password: maktab2025  
Correct! Access granted.
```

Example Run 2 (Lockout):

```
Enter password: test  
Wrong password! Try again.  
Enter password: guess  
Wrong password! Try again.  
Enter password: try  
Wrong password! Try again.  
Too many failed attempts. Account locked!
```

```
# password_validator.py  
# Name: -----  
# Date: -----  
  
# Password validation program  
  
secret_password = "-----" # Your chosen password
```

```

attempts = _____ # Initialize attempt counter

while _____: # What condition?
    user_input = input("Enter password: ")

    if _____: # Check if correct
        _____
        _____
    else:
        _____
        _____

    if _____: # Check if too many attempts
        _____
        _____

```

Save this program as `password_validator.py`

6.2 Option B: Number Guessing Game (20 points)

Create a game where the computer picks a secret number between 1 and 20, and the player keeps guessing until they get it right. Give hints if the guess is too high or too low.

Requirements for Guessing Game

Your program should:

- Set a secret number (between 1 and 20)
- Use a while loop to keep asking for guesses
- Tell player if their guess is too high or too low
- Count the number of guesses
- Congratulate the player when they win

Example Run:

```

I'm thinking of a number between 1 and 20.
Enter your guess: 10
Too low! Try again.
Enter your guess: 15
Too high! Try again.
Enter your guess: 12
Too low! Try again.
Enter your guess: 13
Correct! You got it in 4 guesses!

```

```

# number_game.py
# Name: _____
# Date: _____

# Number guessing game

secret_number = _____ # Pick a number between 1 and 20
guesses = 0

print("I'm thinking of a number between 1 and 20.")

```

```
while _____:          # What condition?
    guess = int(input("Enter your guess: "))
    guesses = _____    # Count this guess

    if _____:          # Check if correct
        _____

    elif _____:         # Check if too low
        _____

    else:                    # Must be too high
        _____
```

Save this program as *number_game.py*

7 Part 7: Reflection Questions (5 points)

Time estimate: 10 minutes

Answer these questions in complete sentences:

1. **Loop Choice Strategy:** When you're deciding between a **for** loop and a **while** loop, what question do you ask yourself first?

2. **The Three Essentials:** Explain why ALL three parts (initialize, condition, update) are necessary for a while loop. What happens if you forget one?

3. **Real-World Application:** Think of a situation in your daily life where a while loop would be useful (something that repeats based on a condition, not a fixed number of times).

8 Debugging Discoveries (5 bonus points)

Growth Mindset: Errors are learning opportunities!

Did you encounter any interesting errors while completing this homework? Share one error and how you fixed it:

My Debugging Story

The Error I Got: _____

What I Tried First: _____

How I Fixed It: _____

What I Learned: _____

No errors? Share a tricky part you figured out instead!

9 Bonus Section: Extra Challenges (Optional - 5 extra points)

Only attempt if you've finished everything else!

9.1 Challenge 1: Nested While Loop Pattern (5 points)

Write a program using nested while loops to print this pattern:

```
*
**
***
****
*****
```

Hint: You'll need an outer loop for rows and an inner loop for stars in each row.

```
# Nested loop pattern
row = 1
while row <= 5:
    # Inner loop for printing stars
    stars = "-----"
    while "-----":
        "-----"
    print() # New line after each row
    "-----" # Update outer loop
```

Save this code as week1_bonus.py

Submission Checklist

Before submitting, make sure you have:

- ☐ Completed all required sections (Parts 1-7)
- ☐ Saved each code exercise in its own file:
 - week1_ex2_1.py (Countdown timer)
 - week1_ex2_2.py (Sum calculator)
 - week1_ex4_1.py (Pattern recognition)

- `week1_ex4_2.py` (Doubling pattern)
- `week1_ex5_1_fixed.py` (Debug: initialization)
- `week1_ex5_2_fixed.py` (Debug: infinite loop)
- `week1_ex5_3_fixed.py` (Debug: condition)
- `password_validator.py` OR `number_game.py` (Creative challenge)

- ☐ Created a folder named `week1_hw_yourname` with all files
- ☐ Tested each code file individually to ensure it runs without errors
- ☐ Checked for infinite loops (does your program eventually stop?)
- ☐ Verified all three parts of each while loop (initialize, condition, update)
- ☐ Completed the memory diagram in Part 3
- ☐ Completed the decision table in Part 1
- ☐ Written your name at the top of this paper
- ☐ Answered all reflection questions
- ☐ Attempted bonus section (optional)

Module 2 Week 1 Reflection:

What was the trickiest part about while loops? _____

Which exercise helped you understand while loops best? _____

Time Tracking:

How long did this homework take you? _____ hours

Parent/Guardian Signature: _____

(Confirming student completed work independently)

Great job exploring while loops!

Next week: Break and Continue - Taking control of your loops